Dr. D. Y. Patil Educational Federation's
DR. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Approved by AICTE, Recognized by Govt. of Maharashtra,
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
Accredited by NAAC with "A" Grade
Web Site: www.dypcoei.edu.in, Email: principal.dypcoei@dypatilef.com

Academic Year: 2026-27

LABORATORY MANUAL

Name of the Student:		
Class: S. Y. B. Tech.	**Division:** A & B	**Roll No.:**
Subject: Data Structures using Artificial Intelligence Laboratory [ADPCC204P/ AMPCC204P]		**PRN No.:**
Department of Artificial Intelligence - Data Science and Machine Learning		

Program Outcomes (PO's):

PO1: **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: **Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4).

PO3: **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: **Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Course Objectives and Course Outcomes (COs)

Course Objectives:

- To develop problem-solving skills by implementing linear and non-linear data structures such as arrays, linked lists, stacks, and queues for real-world applications like job scheduling and appointment systems.
- To analyze and apply different sorting and searching algorithms, and evaluate their time complexity for efficient data organization and processing.
- To understand and implement fundamental AI search algorithms such as BFS and DFS for solving state-space problems like 8-puzzle and water jug problem.
- To design intelligent systems using AI techniques such as game playing, optimization, and decision-making algorithms including Alpha-Beta pruning and TSP.

Course Outcomes:

Course Outcomes: On completion of the course, learners will be able to–

- **CO1:** Apply sorting and searching techniques and analyze their performance using appropriate data structures.
- **CO2:** Implement linear data structures such as arrays, linked lists, stacks, and queues to solve computational problems.
- **CO3:** Apply search algorithms like BFS, DFS, and heuristic techniques to solve AI problems.
- **CO4:** Develop AI-based solutions for classical problems such as games, puzzles, and optimization problems.
- **CO5:** Design and develop real-world applications using data structures and AI concepts with consideration of efficiency and problem-solving.

Dr. D. Y. Patil Educational Federation's

Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION

Department of Artificial Intelligence - Data Science and Machine Learning

Academic Year 2026-27

CERTIFICATE

This is to certify that _____ has successfully completed all the required laboratory experiments for the course **Data Structures using Artificial Intelligence Laboratory,** as prescribed by **Dr. D. Y. Patil College of Engineering & Innovation** an **autonomous college** affiliated to **Savitribai Phule Pune University (SPPU)** for the **Academic Year** 2026-27 (Term-I)**.**

Place:

Date:

Mr. Suraj S. Bhoite/Ms. Sneha Bankar	Dr. Dipannita Mondal	Dr. Suresh Mali
Course In-charge	**Head**	**Director**
Department of AI-DS	**Department, AI-DS & ML**	**DYPCOEI, Varale**

INDEX

Academic Year: 2026-27 **Program Name:** Data Science

Name of the Course: Data Structures using Artificial Intelligence Laboratory

Course Code: ADPCC204P

Class: S.Y. B. Tech. **Div.:** A & B **Semester:** I

Sr. No	Name of Experiment	Date of Conduction	Date of Checking	CO Mapping	Page No	Sign
1.	Write a Python program to store first year percentage of students in array. Write function for sorting array of floating-point numbers in ascending order using i. Selection Sort ii. Bubble Sort and Display Top Five Scores					
2.	Implement a program to perform insertion, deletion, traversal, and searching operations on a one-dimensional array. Analyze the time complexity of each operation.					
3.	Write a program to store the first year percentage of students in an array. Write function for sorting array of floating-point numbers in ascending order using - Selection Sort - Bubble sort and display top five scores.					
4.	Implement a program to perform insertion, deletion, traversal, and searching operations on a one-dimensional array. Analyze the time complexity of each operation.					
5.	Write a program to implement doubly linked list: a) Display free slots b) Book appointment c) Sort list based on time d) Cancel appointment (check validity, time bounds, availability) e) Sort list based on time using pointer manipulation.					

Sr. No.	Name of Experiment	Date of Conduction	Date of Checking	CO Mapping	Page No	Sign
6.	Implement Stack using a linked list. Use this stack to perform evaluation of a postfix expression.					
7.	A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array. Write a program to simulate deque with functions to add and delete elements from either end of the deque.					
8.	Pizza parlor accepting maximum M orders. Orders are served on a first come first served basis. Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write a program for simulating job queue. Write functions to add jobs and delete jobs from the queue.					

Experiment No: 1

To sort First Year Students Percentage Using Selection Sort and Bubble Sort in Python.

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO.1

Title – To sort First Year Students Percentage Using Selection Sort and Bubble Sort in Python.

Aim - To write a Python program to store first-year percentages of students in an array, sort the percentages using Selection Sort and Bubble Sort algorithms, and display the top five scores.

Problem Statement - Write a Python program to store the first-year percentage of students in an array. Implement functions to sort the array of floating-point numbers in ascending order using:

1. Selection Sort
2. Bubble Sort

Prerequisite - Basics of Python Programming, Variables and Data Types, Loops (for, while), Conditional Statements (if), Basic Knowledge of Sorting Algorithms

Software Requirements - Python 3.x or latest, Jupyter Notebook, Visual Studio Code / PyCharm / IDLE, Windows/Linux/macOS Operating System

Learning Objectives –

1. To Understand the concept and importance of sorting algorithms.
2. To Store and manipulate floating-point data using Python lists.
3. To Implement **Selection Sort** to arrange data in ascending order.
4. To Implement **Bubble Sort** to arrange data in ascending order.
5. To Compare the working of Selection Sort and Bubble Sort.
6. To Display the top five highest scores from the sorted list.

Outcomes –

1. Store and manipulate student percentage data using one-dimensional arrays.
2. Implement Selection Sort and Bubble Sort algorithms to arrange data in ascending order.
3. Compare the working and performance of Selection Sort and Bubble Sort based on their time complexity.
4. Analyze and retrieve the top five highest percentages from the sorted dataset

Theory:

Sorting is one of the most fundamental operations in computer science and data structures. It is the process of arranging data elements in a specific order, such as ascending or descending, to facilitate efficient searching, analysis, and data management. Sorting improves the performance of various algorithms, especially searching algorithms like Binary Search, and is widely used in database management systems, file systems, report generation, and data analysis applications.

In this experiment, student percentages are stored in a one-dimensional array and sorted using two comparison-based sorting algorithms: **Selection Sort** and **Bubble Sort**. After sorting the array in ascending order, the top five highest percentages are displayed.

Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list. The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right. This algorithm is not suitable for large data sets as its average and worst-case complexities are of where n is the number of items.

Working of Selection Sort

1. Consider the first element as the minimum.
2. Compare it with all remaining elements.
3. Find the smallest element in the unsorted part.
4. Swap it with the first unsorted element.
5. Move to the next position and repeat the process until the array is completely sorted.

Time Complexity: O(n2) as there are two nested loops.

Algorithm:

```
Algorithm SelectionSort(arr)
    n = length(arr)
    for i = 0 to n - 2
        min_index = i
        for j = i + 1 to n - 1
            if arr[j] < arr[min_index]
                min_index = j
        if min_index != i
            swap(arr[i], arr[min_index])
    return arr
```

Bubble Sort:

It is a simple comparison-based sorting technique. In this sorting technique each element is compared with its adjacent element and the elements are swapped if it's not in order. The time complexity bubble sort is where n is the size of the array. It is not suitable for large data size.

Bubble Sort is another comparison-based sorting algorithm that repeatedly compares adjacent elements and swaps them whenever they are in the wrong order. After each pass, the largest element moves ("bubbles up") to its correct position at the end of the array. This process continues until no more swaps are required, indicating that the array is completely sorted.

Bubble Sort is simple to implement and understand, making it suitable for educational purposes. However, it is inefficient for large datasets because it performs many comparisons and swaps.

Worst and Average Case Time Complexity: O(n*n). Worst case occurs when array is reverse sorted.
Best Case Time Complexity: O(n). Best case occurs when array is already sorted.

Working of Bubble Sort

1. Compare the first two adjacent elements.
2. Swap them if they are in the wrong order.
3. Continue comparing adjacent elements throughout the array.
4. After one complete pass, the largest element reaches its correct position.
5. Repeat the process for the remaining unsorted elements until the array is sorted.

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i=1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i=2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i=3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i=4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i=5	0	1	2	3	4				
	1	1	2	3					
i=6	0	1	2	3					
		1	2						

Algorithm:

```
Algorithm BubbleSort(arr)
    n = length(arr)
    for i = 0 to n - 2
        swapped = False
for j = 0 to n - i - 2
            if arr[j] > arr[j + 1]
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
                swapped = True
        if swapped == False
            break
    return arr
```

Conclusion:

Thus, the program to sort student percentages using Selection Sort and Bubble Sort was successfully implemented and the top five scores were displayed.

Viva Questions:

1. Explain the working of Selection Sort and Bubble Sort?
2. Can Bubble Sort detect an already sorted array? Explain.
3. What happens if duplicate values are present in the array?
4. How can you display the top five highest percentages after sorting?
5. What is the best-case time complexity of Bubble Sort?
6. What is the worst-case time complexity of Selection Sort?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 2

Write a Python Program To Store 10th Class Percentage of Students in Array using Radix sort and Display Top Five Score.

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 2

Title: Write a Python program to store 10th class percentage of students in array. Write function for sorting array of floating-point numbers in ascending order using radix sort and display top five score.

Aim: To write a Python program to store 10th-class percentages of students in an array, sort the percentages using Radix Sort, and display the top five scores.

Problem Statement: Write a Python program to store the 10th-class percentages of students in an array. Implement Radix Sort to sort the percentages and display the top five scores.

Prerequisite: Basics of Python Programming, Arrays (Lists), Functions, Loops and Conditional Statements, Radix Sort

Software Requirements: Python 3.x or latest, Jupyter Notebook, Visual Studio Code / PyCharm / IDLE, Windows/Linux/macOS Operating System

Learning Objectives:

1. Understand the concept of non-comparison-based sorting.
2. Implement the Radix Sort algorithm.
3. Understand the role of Counting Sort in Radix Sort.
4. Analyze the efficiency of Radix Sort.
5. Display the top five highest percentages after sorting.

Outcomes:

1. Store and process student percentage data using arrays.
2. Implement the Radix Sort algorithm to sort numerical data efficiently.
3. Understand the concept of non-comparison-based sorting and digit-wise sorting.
4. Analyze the efficiency of Radix Sort and compare it with comparison-based sorting algorithms.
5. Display the top five highest percentages from the sorted dataset using appropriate data processing techniques.

Theory:

Radix Sort is a **non-comparison-based sorting algorithm** that sorts numerical data by processing individual digits rather than comparing entire numbers. Unlike traditional sorting algorithms such as Bubble Sort, Selection Sort, or Insertion Sort, Radix Sort organizes numbers digit by digit, starting either from the least significant digit (LSD) or the most significant digit (MSD). In this experiment, the **Least Significant Digit (LSD)** approach is used.

Radix Sort processes:

1. Units digit : i.e 1
2. Tens digit : i.e 10
3. Hundreds digit: i.e 100

The algorithm processes each digit position separately using a stable sorting algorithm, typically **Counting Sort**. Since Counting Sort maintains the relative order of equal elements, Radix Sort produces a correctly sorted sequence after processing all digit positions.

In this experiment, student percentages are stored in an array. Since Radix Sort works efficiently with integers, floating-point percentages are first converted into integer values (for example, by multiplying by 100). After sorting, the values are converted back into percentage format, and the top five highest scores are displayed.

Working of Radix Sort

1. Store all percentage values in an array.
2. Convert floating-point percentages into integer values.
3. Identify the maximum element to determine the number of digits.
4. Sort all numbers according to the least significant digit using Counting Sort.
5. Repeat the sorting process for the tens, hundreds, and higher digit places.
6. Continue until all digit positions are processed.
7. Convert the sorted integers back to percentage values.
8. Display the sorted percentages and the top five highest scores.

Algorithm:

- Step 1: Find the maximum element in the array.
- Step 2: Start with the least significant digit (exp = 1).
- Step 3: Apply Counting Sort according to the current digit.
- Step 4: Move to the next digit by multiplying exp by 10.
- Step 5: Repeat until all digits of the maximum element are processed.
- Step 6: The array is sorted.

Pseudocode:

```
RADIX-SORT(A)
    max = maximum element of A
    exp = 1
    while max / exp > 0
        COUNTING-SORT(A, exp)
        exp = exp × 10
```

Conclusion:

Thus, the program to sort student percentages using Radix Sort was successfully implemented, and the top five scores were displayed.

Viva Questions:

1. What is Radix Sort? Explain its working?
2. Where is Radix Sort used in real-world applications?
3. What is the time complexity of Radix Sort?
4. What is the space complexity of Radix Sort?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 3

To sort First Year Students Percentage Using Selection Sort and Bubble Sort in Python

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 3

Title – To sort First Year Students Percentage Using Selection Sort and Bubble Sort in Python.

Aim - To write a Python program to store first-year percentages of students in an array, sort the percentages using Selection Sort and Bubble Sort algorithms, and display the top five scores.

Problem Statement - Write a Python program to store the first-year percentage of students in an array. Implement functions to sort the array of floating-point numbers in ascending order using:

3. Selection Sort
4. Bubble Sort

Prerequisite - Basics of Python Programming, Variables and Data Types, Loops (for, while), Conditional Statements (if), Basic Knowledge of Sorting Algorithms

Software Requirements - Python 3.x or latest, Jupyter Notebook, Visual Studio Code / PyCharm / IDLE, Windows/Linux/macOS Operating System

Learning Objectives –

7. To Understand the concept and importance of sorting algorithms.
8. To Store and manipulate floating-point data using Python lists.
9. To Implement **Selection Sort** to arrange data in ascending order.
10. To Implement **Bubble Sort** to arrange data in ascending order.
11. To Compare the working of Selection Sort and Bubble Sort.
12. To Display the top five highest scores from the sorted list.

Outcomes –

5. Store and manipulate student percentage data using one-dimensional arrays.
6. Implement Selection Sort and Bubble Sort algorithms to arrange data in ascending order.
7. Compare the working and performance of Selection Sort and Bubble Sort based on their time complexity.
8. Analyze and retrieve the top five highest percentages from the sorted dataset

Theory:

Sorting is one of the most fundamental operations in computer science and data structures. It is the process of arranging data elements in a specific order, such as ascending or descending, to facilitate efficient searching, analysis, and data management. Sorting improves the performance of various algorithms, especially searching algorithms like Binary Search, and is widely used in database management systems, file systems, report generation, and data analysis applications.

In this experiment, student percentages are stored in a one-dimensional array and sorted using two comparison-based sorting algorithms: **Selection Sort** and **Bubble Sort**. After sorting the array in ascending order, the top five highest percentages are displayed.

Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list. The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right. This algorithm is not suitable for large data sets as its average and worst-case complexities are of where n is the number of items.

Working of Selection Sort

6. Consider the first element as the minimum.
7. Compare it with all remaining elements.
8. Find the smallest element in the unsorted part.
9. Swap it with the first unsorted element.
10. Move to the next position and repeat the process until the array is completely sorted.

Time Complexity: O(n2) as there are two nested loops.

Algorithm:

```
Algorithm SelectionSort(arr)
    n = length(arr)
    for i = 0 to n - 2
        min_index = i
        for j = i + 1 to n - 1
            if arr[j] < arr[min_index]
                min_index = j
        if min_index != i
            swap(arr[i], arr[min_index])
    return arr
```

Bubble Sort:

It is a simple comparison-based sorting technique. In this sorting technique each element is compared with its adjacent element and the elements are swapped if it's not in order. The time complexity bubble sort is where n is the size of the array. It is not suitable for large data size.

Bubble Sort is another comparison-based sorting algorithm that repeatedly compares adjacent elements and swaps them whenever they are in the wrong order. After each pass, the largest element moves ("bubbles up") to its correct position at the end of the array. This process continues until no more swaps are required, indicating that the array is completely sorted.

Bubble Sort is simple to implement and understand, making it suitable for educational purposes. However, it is inefficient for large datasets because it performs many comparisons and swaps.

Worst and Average Case Time Complexity: O(n*n). Worst case occurs when array is reverse sorted.
Best Case Time Complexity: O(n). Best case occurs when array is already sorted.

Working of Bubble Sort

6. Compare the first two adjacent elements.
7. Swap them if they are in the wrong order.
8. Continue comparing adjacent elements throughout the array.
9. After one complete pass, the largest element reaches its correct position.
10. Repeat the process for the remaining unsorted elements until the array is sorted.

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i=1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i=2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i=3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i=4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i=5	0	1	2	3	4				
	1	1	2	3					
i=6	0	1	2	3					
		1	2						

Algorithm:

```
Algorithm BubbleSort(arr)
    n = length(arr)
    for i = 0 to n - 2
        swapped = False
for j = 0 to n - i - 2
            if arr[j] > arr[j + 1]
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
                swapped = True
        if swapped == False
            break
    return arr
```

Conclusion:

Thus, the program to sort student percentages using Selection Sort and Bubble Sort was successfully implemented and the top five scores were displayed.

Viva Questions:

7. Explain the working of Selection Sort and Bubble Sort?
8. Can Bubble Sort detect an already sorted array? Explain.
9. What happens if duplicate values are present in the array?
10. How can you display the top five highest percentages after sorting?
11. What is the best-case time complexity of Bubble Sort?
12. What is the worst-case time complexity of Selection Sort?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 4

Implement A Program to Perform Insertion, Deletion, Traversal, And Searching Operations on A One-Dimensional Array. Analyze The Time Complexity of Each Operation

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 4

Title: Implement a program to perform insertion, deletion, traversal, and searching operations on a one-dimensional array. Analyze the time complexity of each operation.

Aim: To write a Python program to perform **Insertion, Deletion, Traversal, And Searching Operations** on a one-dimensional array and analyze the time complexity of each operation.

Problem Statement: Write a Python program to create a one-dimensional array and implement basic array operations such as insertion, deletion, traversal, and searching. Analyze the efficiency of each operation using time complexity.

Prerequisite: Basics of Python Programming, Arrays (Lists), Functions, Loops and Conditional Statements, Data Structures, Searching Techniques, and Time Complexity Analysis.

Software Requirements: Python 3.x or latest, Jupyter Notebook, Visual Studio Code / PyCharm / IDLE, Windows/Linux/macOS Operating System

Learning Objectives:

1. Understand the concept of one-dimensional arrays.
2. Implement insertion and deletion operations on arrays.
3. Perform traversal and searching operations on array elements.
4. Analyze the time complexity of different array operations.
5. Understand the importance of efficient data handling using arrays.

Outcomes:

1. Store and process data using one-dimensional arrays.
2. Implement insertion, deletion, traversal, and searching operations.
3. Understand the working of array manipulation techniques.
4. Analyze the efficiency of array operations using Big-O notation.
5. Apply array operations in real-world data processing applications.

Theory:

A **one-dimensional array** is a linear data structure that stores a collection of elements of the same data type in contiguous memory locations. Each element in an array can be accessed using its index value. Arrays allow various operations such as insertion, deletion, traversal, and searching.

1. Insertion Operation

Insertion is the process of adding a new element at a specified position in an array. To insert an element, the existing elements after that position are shifted one position towards the right.

2. Deletion Operation

Deletion removes an element from a specified position in an array. After deletion, the remaining elements are shifted towards the left to fill the empty space.

3. Traversal Operation

Traversal is the process of accessing and displaying each element of an array one by one.

4. Searching Operation

Searching is used to find the location of a specific element in an array. Linear Search is commonly used for unsorted arrays, where each element is checked sequentially.

Working of Array Operations:

1. Create an array to store elements.
2. Insert elements at required positions by shifting existing elements.
3. Delete elements by removing the required value and shifting remaining elements.
4. Traverse the array to display all stored elements.
5. Search for a specific element by comparing each array value.
6. Display the result of each operation.

Algorithm:

1. Start.
2. Create an empty array.
3. Read the elements and store them in the array.

4. **For Insertion:**
   - Start.
   - Read the array.
   - Read the element to be inserted.
   - Read the position.
   - Check whether the position is valid.
   - Insert the element at the specified position.
   - Display the updated array.
   - Stop.
5. **For Deletion:**
   - Start.
   - Read the array.
   - Read the position of the element to be deleted.
   - Check whether the position is valid.
   - Delete the element.
   - Display the updated array.
   - Stop.
6. **For Traversal:**
   - Start.
   - Read the array.
   - Set `i = 0`.
   - Repeat until `i < n`.
   - Display `A[i]`.
   - Increment `i`.
   - Stop.
7. **For Searching:**
   - Start.
   - Read the array.
   - Set i = 0.
   - Repeat until i < n.
   - Display A[i].
   - Increment i.
   - Stop.

Space Complexity:

- The space complexity of storing elements in an array is **O(n)**.
- Extra memory used during operations is **O(1)**.

Time Complexity Analysis:

Operation	Best Case	Worst Case	Explanation
Insertion	O(1)	O(n)	Inserting at the end requires no shifting, while insertion at the beginning requires shifting elements.
Deletion	O(1)	O(n)	Deleting the last element is faster, while deleting from the beginning requires shifting.
Traversal	O(n)	O(n)	Every element must be visited once.
Searching	O(1)	O(n)	Element may be found at the first position or may require checking the entire array.

Conclusion:

Thus, the Python program to perform **insertion, deletion, traversal, and searching operations on a one-dimensional array** was successfully implemented. The time complexity of each operation was analyzed and understood.

Viva Questions:

1. What is a one-dimensional array?
2. Explain the insertion operation in an array.
3. What happens internally during deletion of an array element?
4. What is array traversal?
5. Which searching technique is suitable for an unsorted array?
6. What is the time complexity of insertion in an array?
7. What is the time complexity of searching in an array?
8. What is the difference between an array and a linked list?
9. What is the space complexity of an array?
10. Why is shifting required during insertion and deletion operations?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 5

To Implement a **Doubly Linked List**

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 5

Title: To Implement a **Doubly Linked List** to perform the following operations:

1. Display free slots
2. Book appointment
3. Sort list based on time
4. Cancel appointment (check validity, time bounds, availability)

   Sort list based on time using pointer manipulation.

Aim: To implement a Doubly Linked List for appointment scheduling, including displaying free slots, booking, and cancelling appointments. To sort appointments based on time using pointer manipulation.

Problem Statement: To implement a **Doubly Linked List in Python** for appointment scheduling and perform the following operations:

1. Display free slots.
2. Book an appointment.
3. Sort the appointment list based on time using pointer manipulation.
4. Cancel an appointment by checking its validity, time bounds, and availability.

Prerequisite:

- Basic knowledge of Python programming.
- Understanding of classes and objects.
- Knowledge of pointers/references in linked data structures.
- Basic understanding of linked lists.

Software Requirements:

- **Programming Language:** Python 3.x
- **IDE/Editor:** IDLE, Visual Studio Code, PyCharm, or Jupyter Notebook
- **Operating System:** Windows, Linux, or macOS

Learning Objectives:

- To understand the concept and structure of a Doubly Linked List.
- To implement a Doubly Linked List using Python classes and objects.
- To perform appointment booking and cancellation operations.
- To display available appointment slots.
- To sort appointments based on time using node reference manipulation.

Outcomes:

After completing this experiment, students will be able to:

1. Create and implement a Doubly Linked List in Python.
2. Display free appointment slots.
3. Book an appointment after checking slot availability.
4. Cancel an appointment after validating the appointment and time bounds.
5. Sort appointment slots based on time by manipulating node links.

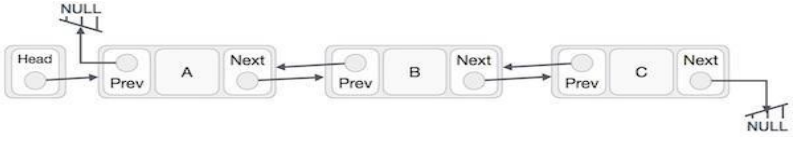
Theory:

A Doubly Linked List (DLL) is a linear data structure in which each node contains:

- **Data:** Stores the appointment information, such as time and booking status.
- Previous (Prev) Reference: Stores a reference to the previous node.
- Next (next) Reference: Stores a reference to the next node.

  Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, forward as well as backward easily as compared to Single Linked List. Following are the important terms to understand the concept of doubly linked list.
- **Link** − Each link of a linked list can store a data called an element.
- **Next** − Each link of a linked list contains a link to the next link called Next.
- **Prev** − Each link of a linked list contains a link to the previous link called Prev.
- **Linked List** − A Linked List contains the connection link to the first link called First and to the last link called Last.

Doubly Linked List Representation



As per the above illustration, following are the important points to be considered.

- Doubly Linked List contains a link element called first and last.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Each link is linked with its previous link using its previous link.
- The last link carries a link as null to mark the end of the list.

Basic Operations in Doubly Linked List

Following are the basic operations supported by a list.

- Insertion − Adds an element at the beginning of the list.
- Insert Last − Adds an element at the end of the list.
- Insert After − Adds an element after an item of the list.
- Deletion − Deletes an element at the beginning of the list.
- Delete Last − Deletes an element from the end of the list.
- Delete − Deletes an element from the list using the key.
- Display forward − Displays the complete list in a forward manner.
- Display backward − Displays the complete list in a backward manner.

Algorithm:

1. Start
2. Create a Node class with:
   time, status, prev, next
3. Create a Doubly Linked List.
4. Add appointment slots to the list.
5. Display all slots where:
   status == "Free"
6. Input appointment time for booking.
7. If time is within valid bounds:
   Search for the slot.
   If slot.status == "Free":
   slot.status = "Booked"
   Else:

Display "Slot unavailable"

Else:

Display "Invalid appointment time"

8. Sort the list based on time by changing:

prev and next references

9. Input appointment time for cancellation.

10. If time is within valid bounds:

Search for the appointment.

If appointment.status == "Booked":

appointment.status = "Free"

Else:

Display "Invalid appointment"

Else:

Display "Invalid appointment time"

11. Display the updated appointment list.

12. Stop

Pseudocode:

```
def create_slot(time):
    Create a new node with time, status, prev, and next
def display_free_slots():
    Traverse the Doubly Linked List
    Display slots with status = "Free"
def book_appointment(time):
    If time is within valid bounds:
        Search for the slot
        If slot is Free:
            Change status to "Booked"
        Else:
            Display "Slot unavailable"
def sort_by_time():
```

Traverse the Doubly Linked List

Compare appointment times

Rearrange nodes using prev and next references

def cancel_appointment(time):

If time is within valid bounds:

Search for the appointment

If status is "Booked":

Change status to "Free"

Else:

Display "Invalid appointment"

Conclusion:

Thus, a **Doubly Linked List was successfully implemented in Python** for appointment scheduling. The program performs operations such as displaying free slots, booking appointments, cancelling appointments after checking validity and availability, and sorting appointments based on time using node reference manipulation.

Viva Questions:

1. What is a Doubly Linked List?
2. What is the difference between a Singly Linked List and a Doubly Linked List?
3. What are the components of a node in a Doubly Linked List?
4. What is the purpose of prev and next in a Doubly Linked List?
5. Why is a Doubly Linked List used for appointment scheduling?
6. How do you create a node in Python?
7. How do you create a Doubly Linked List in Python?
8. What is the purpose of the head pointer?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charg
5	3	2	10	

Experiment No: 6

To Implement Stack Using a Linked List and Use the Stack to Perform Evaluation of a Postfix Expression.

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 6

Title: To Implement Stack Using a Linked List and Use the Stack to Perform Evaluation of a Postfix Expression.

Aim: To write a Python program to implement a Stack using a Linked List and use the stack to evaluate a given postfix expression.

Problem Statement: Write a Python program to implement a stack using a linked list. The stack should support the following operations:

1. Push an element onto the stack.
2. Pop an element from the stack.
3. Display the elements of the stack.
4. Evaluate a postfix expression using the implemented stack.
5. Display the final result of the postfix expression.

Prerequisite: Basics of Python Programming, Variables and Data Types, Conditional Statements, Loops, Functions, Basic understanding of Classes and Objects, Basic knowledge of Linked Lists, Basic knowledge of Stack Data Structure, Understanding of Arithmetic Operators

Software Requirements: Python 3.x or latest, Jupyter Notebook, Visual Studio Code / PyCharm / IDLE, Windows/Linux/macOS Operating System

Learning Objectives:

1. To understand the concept and applications of the **Stack data structure**.
2. To understand the **Last-In-First-Out (LIFO)** principle.
3. To implement a stack using a **linked list** in Python.
4. To implement basic stack operations such as **Push, Pop, Peek, and Display**.
5. To understand the concept of **postfix expressions**.
6. To evaluate a postfix expression using a stack.

Outcomes:

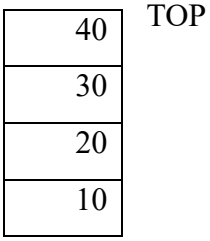
After successful completion of this experiment, the student will be able to:

1. Implement a Stack using a Linked List, the working principle and applications of a Stack.
2. Perform Push, Pop, Peek, and Display operations on a Stack.
3. Evaluate arithmetic expressions represented in postfix notation using a Stack.
4. Handle stack overflow/underflow conditions appropriately.
5. Analyze the time and space requirements of stack operations.
6. Apply Stack data structures to solve practical computational problems.

Theory –

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. When implementing a stack via a linked list, elements are inserted and removed from the head of the list, which serves as the top. This ensures that both push and pop operations execute in O(1) time complexity.

For example:



Here, 40 is the top element. If a pop operation is performed, 40 will be removed first.

Stack Operations -

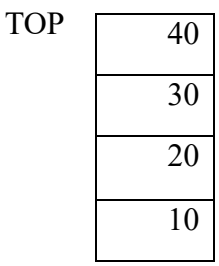
1. Push

The **Push** operation inserts a new element at the top of the stack.

Push(item): Create a new node. Set its next pointer to top. Update top to point to this new node.

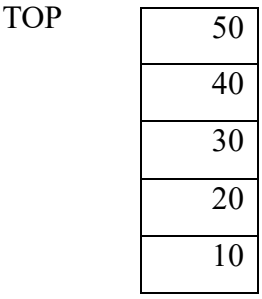
Example:

Before Push:



Push (50)

After Push:



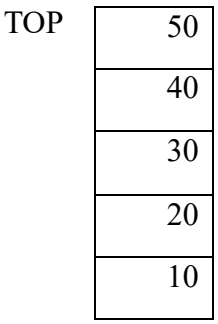
2. Pop

The **Pop** operation removes and returns the element from the top of the stack.

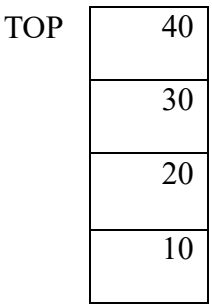
Pop(): If top is null, output underflow. Otherwise, store the data of top, advance top to top->next, free the old node, and return the data.

Example:

Before Pop:



After Pop:



The removed element is 50.

3. Peek

The **Peek** operation returns the top element without removing it from the stack.

4. Display

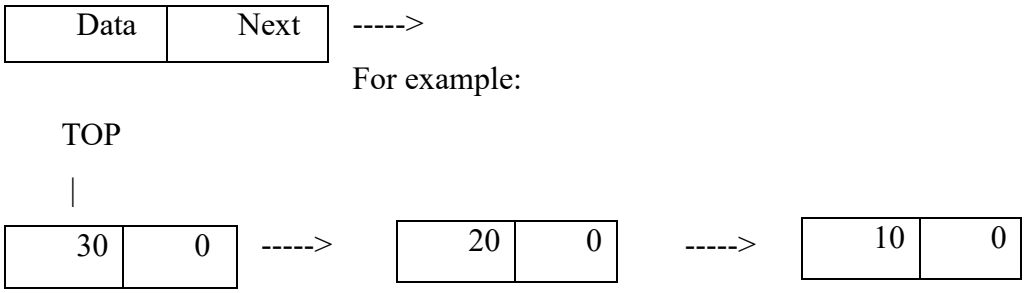
The **Display** operation shows all the elements currently present in the stack.

Linked List

A **Linked List** is a dynamic linear data structure consisting of nodes. Each node contains:

1. **Data** – stores the value.
2. **Next** – stores the reference/address of the next node.

A simple linked list can be represented as:



When a stack is implemented using a linked list, the **top of the stack is maintained at the beginning of the linked list**. Therefore, Push and Pop operations can be performed efficiently.

Stack Using Linked List -

In a linked-list implementation of a stack:

- The top pointer/reference points to the first node.
- During **Push**, a new node is created and inserted at the beginning.
- During **Pop**, the first node is removed.
- If top is None, the stack is empty.

Algorithm

Algorithm for Implementing Stack Using Linked List

Step 1: Start.

Step 2: Define a Node class containing data and next.

Step 3: Define a Stack class and initialize top = None.

Step 4: For the Push operation:

- Create a new node.
- Store the given data in the node.
- Set the new node's next to top.
- Update top to the new node.

Step 5: For the Pop operation:

- Check whether top is None.
- If yes, display "Stack Underflow".
- Otherwise, store the top element.
- Move top to the next node.
- Return the removed element.

Step 6: For the Peek operation:

- Check whether the stack is empty.
- If not empty, return the element at top.

Step 7: For the Display operation:

- Start from top.
- Traverse the linked list until None.
- Display each element.

Step 8: Stop.

Algorithm for Postfix Evaluation

Step 1: Start.

Step 2: Read the postfix expression.

Step 3: Create an empty stack.

Step 4: Scan the postfix expression from left to right.

Step 5: If the current token is an operand, push it onto the stack.

Step 6: If the current token is an operator:

- Pop the second operand from the stack.
- Pop the first operand from the stack.

- Perform the required operation.
- Push the result onto the stack.

Step 7: Repeat Steps 5 and 6 until all tokens are processed.

Step 8: Pop the final value from the stack.

Step 9: Display the result.

Step 10: Stop.

Time Complexity:

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)
Display	O(n)
Postfix Evaluation	O(n)

The space complexity of the stack is **O(n)**, where n is the number of elements/tokens stored in the stack.

Conclusion:

Thus, the program to **implement a Stack using a Linked List** was successfully developed. The Push, Pop, Peek, and Display operations were implemented successfully. The implemented stack was then used to **evaluate a postfix expression**, and the final result was obtained successfully.

Viva Questions:

1. What is a Stack? Explain the LIFO principle.
2. What is the difference between a Stack and a Queue?
3. Why is a Stack called a LIFO data structure?
4. What is the purpose of the top pointer in a Stack?
5. Explain the Push and Pop operations.
6. What is Stack Underflow?
7. What is the advantage of implementing a Stack using a Linked List?
8. What is the time complexity of Push and Pop operations in a linked-list implementation?

9. What is a postfix expression?
10. What is the difference between infix, prefix, and postfix expressions?
11. Why is a Stack used for postfix expression evaluation?
12. How is the order of operands maintained while evaluating subtraction and division?
13. Evaluate the postfix expression 5 6 2 + *.
14. What happens if an operator is encountered when fewer than two operands are available in the Stack?
15. What is the time complexity of postfix expression evaluation?
16. What is the space complexity of postfix expression evaluation?
17. What is the difference between an array-based Stack and a linked-list-based Stack?
18. What are the applications of Stack in real-world computing?
19. Can a Stack implemented using a Linked List have a fixed size?
20. What happens when the Stack is empty and a Pop operation is performed?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charg
5	3	2	10	

Experiment No: 7

To Implement Implementation of Double-Ended Queue (Deque) Using a One-Dimensional Array in Python.

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 7

Title: To Implement Implementation of Double-Ended Queue (Deque) Using a One-Dimensional Array in Python.

Aim: To implement a **Double-Ended Queue (Deque)** using a one-dimensional array and perform insertion and deletion operations from both ends.

Problem Statement: To design and implement a Deque using a one-dimensional array in Python. The program should allow the user to add and delete elements from both the **front** and **rear** ends of the deque.

Prerequisite:

- Basic knowledge of Python programming.
- Understanding of arrays/lists.
- Knowledge of functions, conditions, and loops.
- Basic understanding of queue data structures.

Software Requirements:

- **Programming Language:** Python 3.x
- **IDE/Editor:** IDLE, Visual Studio Code, PyCharm, or Jupyter Notebook
- **Operating System:** Windows, Linux, or macOS

Learning Objectives:

- To understand the concept of a Double-Ended Queue.
- To implement a Deque using a one-dimensional array.
- To perform insertion and deletion from both ends.
- To understand overflow and underflow conditions.

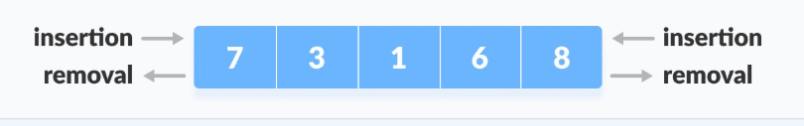
Outcomes:

After completing this experiment, students will be able to:

- Implement a Deque using Python.
- Add elements from the front and rear.
- Delete elements from the front and rear.
- Check whether the Deque is empty or full.
- Understand the practical application of a Deque.

Theory:

Deque or Double Ended Queue is a type of queue in which insertion and removal of elements can either be performed from the front or the rear. Thus, it does not follow **FIFO rule (First In First Out).**



Representation of Deque

Type of Deque:

1. **Input restricted Deque**

   In this deque, input is restricted at a single end but allows deletion at both the ends.

2. **Output Restricted Deque**

   In this deque, output is restricted at a single end but allows insertion at both the ends.

Operations on a Deque:

In a linear array implementation, if the array is full, no more elements can be inserted. In each of the operations below, if the array is full, "overflow message" is thrown.

Before performing the following operations, these steps are followed.

1. Take an array (deque) of size n.
2. Set two pointers front = -1 and rear = 0.

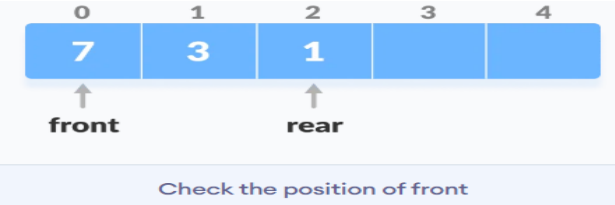


Initialize an array and pointers for deque

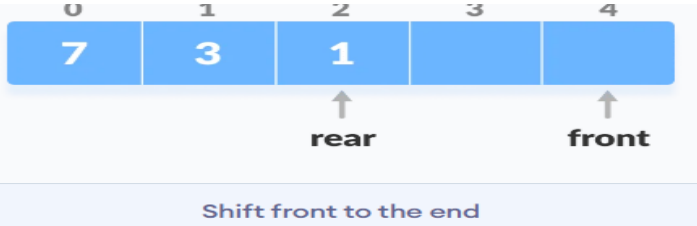
1. Insert at the Front

This operation adds an element at the front.

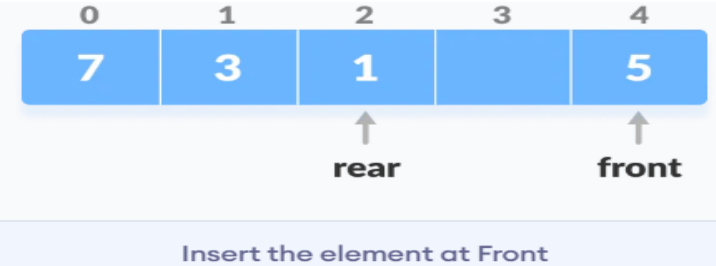
1. Check if the deque is full.



2. If the deque is full (i.e. (front == 0 && rear == n - 1) || (front == rear + 1)), insertion operation cannot be performed (**overflow condition**).
3. If the deque is empty, reinitialize front = 0. And, add the new key into array[front].
4. If front = 0, reinitialize front = n-1 (last index).



5. Else, decrease front by 1.
6. Add the new key 5 into array[front].

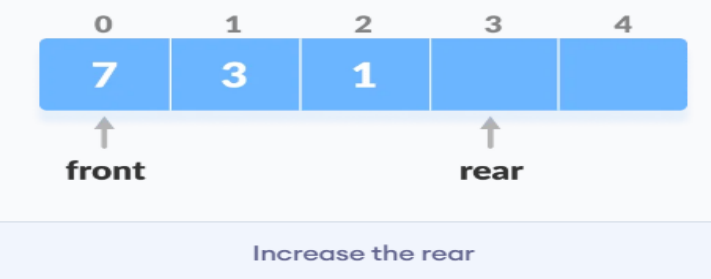


2. Insert at the Rear

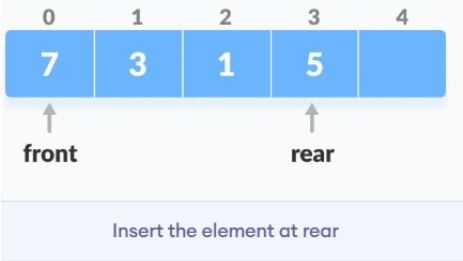
This operation adds an element to the rear.

1. Check if the deque is full.
2. If the deque is full, insertion operation cannot be performed (**overflow condition**).

3. If the deque is empty, reinitialize rear = 0. And, add the new key into array[rear].
4. If rear = n - 1, reinitialize real = 0 (first index).
5. Else, increase rear by 1.



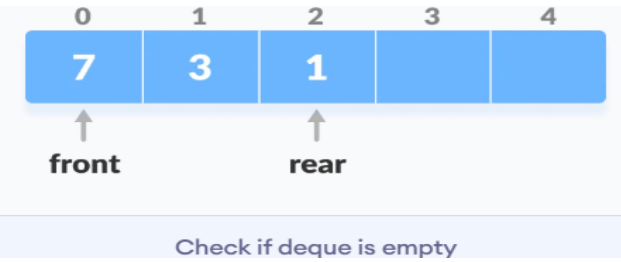
6. Add the new key 5 into array[rear].



3. Delete from the Front

The operation deletes an element from the front.

1. Check if the deque is empty.
2. If the deque is empty (i.e. front = -1), deletion cannot be performed (**underflow condition**).
3. If the deque has only one element (i.e. front = rear), set front = -1 and rear = -1.
4. Else if front is at the last index (i.e. front = n - 1), set front = 0.
5. Else, front = front + 1.





4. Delete from the Rear

This operation deletes an element from the rear.

1. Check if the deque is empty



1. If the deque is empty (i.e. front = -1), deletion cannot be performed (**underflow condition**).
2. If the deque has only one element (i.e. front = rear), set front = -1 and rear = -1, else follow the steps below.
3. If rear is at the first index (i.e. rear = 0), reinitialize rear = n - 1.
4. Else, rear = rear - 1



5. Check Empty

This operation checks if the deque is empty. If front = -1, the deque is empty.

6. Check Full

This operation checks if the deque is full. If front = 0 and rear = n - 1 OR front = rear + 1, the deque is full.

Time Complexity

The time complexity of all the above operations is constant i.e. **O(1).**

Basic Operations

- Add Front: Insert an element at the front.
- Add Rear: Insert an element at the rear.
- Delete Front: Remove an element from the front.
- Delete Rear: Remove an element from the rear.
- Display: Display all elements of the Deque.

Algorithm:

1. Start.
2. Create an empty Python list deque with a maximum size.
3. Define add_front() and check if the Deque is full.
4. If not full, insert the element at the front.
5. Define add_rear() and check if the Deque is full.
6. If not full, insert the element at the rear.
7. Define delete_front() and check if the Deque is empty.
8. If not empty, remove the front element.
9. Define delete_rear() and check if the Deque is empty.
10. If not empty, remove the rear element.
11. Define display() to show all Deque elements.
12. Perform the selected Deque operation.
13. Repeat until the user selects Exit.
14. Stop.

Pseudocode

```
def add_front(value):
    if deque is full:
        display "Deque Overflow"
    else:
        insert value at front
def add_rear(value):
    if deque is full:
```

```
        display "Deque Overflow"
    else:
        insert value at rear
def delete_front():
    if deque is empty:
        display "Deque Underflow"
    else:
        delete element from front
def delete_rear():
    if deque is empty:
        display "Deque Underflow"
    else:
        delete element from rear
def display():
    display all elements of deque
```

Conclusion:

Thus, a Double-Ended Queue (Deque) was successfully implemented using a one-dimensional array in Python, and insertion and deletion operations were performed from both the front and rear ends.

Viva Questions:

1. What is a Deque?
2. What is the full form of Deque?
3. What is the difference between a queue and a Deque?
4. From how many ends can insertion be performed in a Deque?
5. From how many ends can deletion be performed in a Deque?
6. What is the difference between add_front() and add_rear()?
7. What is the difference between delete_front() and delete_rear()?
8. What is an array representation of a Deque?
9. What is Deque overflow?

Coding Efficien	Viva	Timely Comple	Total	Dated Sign of Course In-charg
5	3	2	10	

Experiment No: 8

Pizza Parlor Accepting Maximum M Orders

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Date: **Mark:** /10

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 8

Title: Pizza parlor accepting maximum M orders. Orders are served on a first come first served basis. Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write a program for simulating job queue. Write functions to add jobs and delete jobs from the queue.

Aim: To implement a queue data structure to simulate a pizza parlor accepting a maximum of M orders, where orders are served on a First Come First Served (FCFS) basis. Implement functions to add (enqueue) and delete (dequeue) jobs from the queue.

Problem Statement: A pizza parlor accepts a maximum of M orders. The orders are served in the same order in which they are received, i.e., First Come First Served (FCFS).
A queue is a suitable data structure for this problem because it follows the FIFO (First In, First Out) principle. The program should provide the following operations:

1. Add a new order/job to the queue.
2. Delete/serve the first order/job from the queue.
3. Display all waiting orders/jobs.
4. Check whether the queue is full or empty.

Prerequisite:

- Basic knowledge of Python programming.
- Understanding of arrays/lists.
- Knowledge of functions, conditions, and loops.
- Basic understanding of queue data structures.

Software Requirements:

- **Programming Language:** Python 3.x
- **IDE/Editor:** IDLE, Visual Studio Code, PyCharm, or Jupyter Notebook
- **Operating System:** Windows, Linux, or macOS

Learning Objectives:

- Understand the concept of Queue and its FIFO (First In, First Out) principle.
- Understand the real-world application of queues through the Pizza Parlor/Job Queue simulation.
- Implement queue operations such as enqueue (add) and dequeue (delete/serve).
- Develop functions in Python for insertion, deletion, and display of queue elements.
- Handle queue conditions such as overflow (queue full) and underflow (queue empty).

Outcomes:

- Understand the concept of a Queue and the FIFO (First In, First Out) principle.
- Implement a queue with a maximum capacity of M using Python.
- Perform basic queue operations such as enqueue (add) and dequeue (delete/serve).
- Develop a program to simulate FCFS (First Come First Served) job/order processing.
- Handle queue overflow and underflow conditions effectively.

Theory:

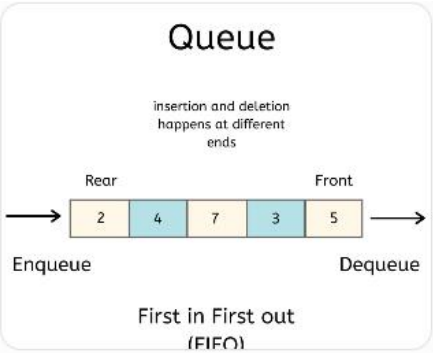
A **Queue** is a linear data structure in which insertion of an element takes place at one end called the **REAR**, and deletion takes place at the other end called the **FRONT**. A queue follows the **FIFO (First In, First Out)** principle. This means that the element inserted first will be removed first.

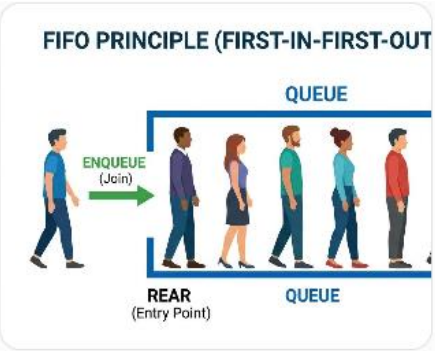
For example, in a pizza parlor, if customers place orders in the following sequence:

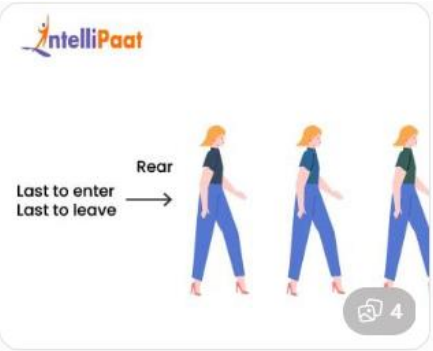
Order 1 → Order 2 → Order 3 → Order 4

then the orders will be served in the same sequence:

Order 1 → Order 2 → Order 3 → Order 4







1. Queue Structure

A queue mainly consists of two pointers/indexes:

- **FRONT:** Indicates the position of the first element in the queue.
- **REAR:** Indicates the position where the next element will be inserted.

2. Enqueue Operation

Enqueue is the operation of adding a new element to the **REAR** of the queue.

3. Dequeue Operation

Dequeue is the operation of removing an element from the **FRONT** of the queue.

4. Job Queue Simulation

In an operating system, multiple jobs may arrive for processing. If the system does not assign priorities to jobs, they are processed according to their arrival time.

5. Pizza Parlor Application

A pizza parlor can accept a maximum of **M orders**. Each new order is added to the rear of the queue. When an order is ready to be served, the order at the front is removed.

Complexity:

For a properly implemented queue:

- **Enqueue:** O(1)
- **Dequeue:** O(1)
- **Display/Traversal:** O(n)
- **Space Complexity:** O(M)

Algorithm:

Algorithm: ENQUEUE

1. Check whether the queue is full.
2. If the queue is full, display **"Queue Overflow"**.
3. Otherwise, increase rear by 1.
4. Insert the new job/order at queue[rear].
5. If this is the first element, set front = 0.
6. Display confirmation message.

Algorithm: DEQUEUE

1. Check whether the queue is empty.
2. If the queue is empty, display **"Queue Underflow"**.
3. Otherwise, retrieve the element at queue[front].
4. Increase front by 1.
5. If the queue becomes empty, reset front and rear.
6. Display the served job/order.

Algorithm: DISPLAY

1. Check whether the queue is empty.
2. If empty, display **"Queue is Empty"**.
3. Otherwise, traverse the queue from front to rear.
4. Display all waiting jobs/orders.

Time Complexity:

1. Add / Enqueue: O(1)
2. Delete / Dequeue: O(1)
3. Display: O(n)
4. Check Full/Empty: O(1)

Space Complexity:

The queue can store a maximum of **M orders**, therefore:

Space Complexity = O(M)

Conclusion:

The **Pizza Parlor/Job Queue** was successfully implemented using the **Queue data structure** in Python. The queue follows the **FIFO principle**, ensuring that orders are served in the same sequence in which they are received. The program demonstrates **enqueue, dequeue, and display** operations and handles both **queue overflow and underflow** conditions.

Viva Questions:

1. What is a Queue?
2. What does FIFO mean?
3. What are the two ends of a Queue?
4. Where is insertion performed in a Queue?
5. Where is deletion performed in a Queue?
6. What is Enqueue and Dequeue?
7. What is Queue Overflow?
8. What is Queue Underflow?
9. What is the time complexity of Enqueue and Dequeue?
10. What is the difference between Stack and Queue?

Coding Efficien	Viva	Timely Comple	Total	Dated Sign of Course In-charg
5	3	2	10	